Agent tooling decision matrix: Claude Code vs Codex CLI vs an AIDA-style substrate

AIDA positioning · best-effort comparison — live source: github.com/joemooney/aida

rendered 2026-07-10

Last updated: 2026-07-09

This is a decision-support page, not a sales page. It is for a team or an individual sitting in front of the "which agent tooling, and how much of it" question: stay in Claude Code, move to Codex CLI, or push some of the workflow down into vendor-neutral project infrastructure (scripts, CI, git hooks, MCP servers, or a substrate like AIDA).

The honest answer for many readers is **"you do not need a substrate at all"** or **"stay where you are."** The matrix below is built to tell you that when it is true. A decision aid that only ever points at adopting more tooling is a sales pitch wearing a table's clothes; the value here is the rows that send you the other way.

Every row is grounded in trade-offs already documented in the migration material, not in aspiration. The source documents are:

- `docs/agents/porting-claude-code-to-codex.md` — the conceptual Claude-to-Codex migration analysis (what ports cleanly, what does not, and why).
- `docs/agents/claude-surfaces-codex-parity.md` — the ground-truth surface-by-surface inventory (file and symbol references for every Claude-specific hook, hook event, status line, headless path, and TUI dependency).
- `docs/plans/2026-06-25-codex-defer-resume-design.md` — the escalation/resume design.
- `docs/plans/2026-06-25-codex-sandbox-posture-design.md` — the sandbox design.

Both CLIs move fast. Re-verify any capability claim against your installed version before making a binding decision; the source docs above carry their own "verified on " stamps for exactly that reason.

The shape of the decision

There are really three separate questions hiding inside "which agent tooling," and conflating them is the most common mistake:

1. **Which agent runtime do I run day to day?** Claude Code, Codex CLI, both, or something else. This is a tooling-and-taste choice, and it is reversible if you set it

up to be.

2. **Where do my workflow guarantees live?** Inside one vendor's agent runtime (its hooks, its skills, its status line, its memory), or outside it (scripts, CI, git hooks, MCP servers, durable docs). This is the choice that actually determines lock-in.
3. **Do I need a coordination substrate on top of all that?** A durable, shared, queryable layer of project intent and multi-agent state — or is a good `AGENTS.md` plus issue tracker plus scripts enough?

Most readers should spend their attention on question 2 and answer question 3 "no, not yet." The matrix keeps the three questions separate on purpose.

The one-line tests

Three quick filters before the detailed table. If all three point the same way, you can usually stop reading.

- **Reversibility test:** *If a Claude Code (or Codex) feature vanished tomorrow, would a correctness guarantee vanish with it?* If yes, that guarantee is in the wrong place — move it outward (CI, git hook, sandbox, MCP server) regardless of which agent you run. This is the single most important takeaway and it does not require any substrate at all.
- **Cross-vendor test:** *Will more than one agent vendor ever touch the same project state?* If no, vendor-neutral infrastructure (including a substrate) earns far less. A single-vendor shop should lean into that vendor's native coordination, not pay for portability it will not use.
- **Compounding test:** *Does value compound — many specs, many sessions, many agents, many months, many hands?* If no, a substrate is overhead without the payoff. A good `AGENTS.md` and an issue tracker are the right altitude.

Axis-by-axis matrix

Each row is one capability axis. The three tooling columns describe where that capability is strongest; the final column is the default recommendation for a *reversible* setup. Where a claim comes straight from a source doc, the doc is named inline.

Axis	Claude Code	Codex CLI	AIDA-style substrate	Default for a reversible setup
Hooks (lifecycle events)	Broadest lifecycle event set; richer handler types (command, HTTP, MCP, prompt, agent)	Narrower but real event set (PreToolUse, PermissionRequest, PostToolUse, Pre/PostCompact, UserPromptSubmit, SessionStart, Stop, SubagentStart/Stop); command handlers; as of 2026 can mutate tool I/O via `updatedInput` / result replacement	n/a — a substrate does not host hooks	Classify each hook by purpose first (observation / guidance / enforcement / mutation / approval). Observation and guidance port cleanly to either agent. Enforcement should not live in an agent hook at all (next row).
Enforcement (blocking unsafe work)	Hook can block selected events; per-matcher PreToolUse blocking	Hook can deny; sandbox + approval profiles are the stronger native boundary	Substrate gate (MCP/CLI refuses the unsafe op), git hooks, CI	**Move enforcement OUT of the agent runtime.** A git pre-commit/pre-push hook, CI check, sandbox profile, or an MCP server that refuses the operation is the portable boundary. An agent hook is at best a redundant early warning. (porting doc, “Treat hooks as similar shape, different contract”)

Axis	Claude Code	Codex CLI	AIDA-style substrate	Default for a reversible setup
Approval / defer semantics	Pending-tool-call `defer`: pause one specific tool call, let an external authority decide, resume the *same* call via `--resume`. The clearest Claude-only primitive	No tool-call `defer`. Has turn-level continuation (`Stop {deci-sion:"block"}`) and session-level resume (`codex exec resume`), which is coarser	"Park, decide out of band, resume a fresh run with the decision in context" — durable punt/finding/direction state, agent-agnostic	Build approval as **durable external state**, not an implicit paused turn. The substrate shape (park + resume with context) is what survives a vendor switch; `defer` is an optional accelerator, not a foundation. (defer/resume design, sections 2 and 7)

Axis	Claude Code	Codex CLI	AIDA-style substrate	Default for a reversible setup
Sandboxing	Native sandbox confines the Bash tool; Edit/Write/MCP and the process itself are not confined by it — a real gap an outer wrapper (bubblewrap) fills	Stronger native OS sandbox: Landlock + seccomp on Linux, Seatbelt on macOS, restricted tokens on Windows; whole process tree; **egress blocked by default** under `workspace-write`; cross-platform	n/a for confinement itself; a substrate can *select and verify* the right native profile and fail closed	Prefer the **vendor's native sandbox** where it is strong (Codex), add an outer OS wrapper only to fill a real gap (Claude's tool-only sandbox). Do not nest a weaker wrapper around a stronger native one. State explicitly whether the boundary is read, write, or network confinement, and fail closed. (sandbox design, sections 1, 5, 6)

Axis	Claude Code	Codex CLI	AIDA-style substrate	Default for a reversible setup
Status / observability	Command-backed status line (`statusLine.command` can run e.g. `aida statusline`) — live, scriptable footer	Built-in TUI footer with a fixed (rich) field set; **no command-backed status line yet** (open upstream FR); `codex exec --json` gives a structured event stream for automation	`aida status` polling, a TUI, or shell-prompt/tmux integration — agent-independent	This is **UX, not correctness.** Losing the command-backed footer in Codex is a real but low-severity gap; rebuild the readout in the shell prompt, tmux, a TUI, or by polling. Do not let an observability nicety drive a tooling decision. (parity inventory, section 2; porting doc, "the one genuine gap")
Slash commands	Rich `.claude/commands` custom slash-command files; strong muscle memory	**No custom slash-command files.** Maps to Codex skills, built-in slash commands, or project scripts (`make`/`just`/`npm run`)	State-changing workflows live as CLI verbs both humans and agents run	If a command changes project state, back it with a **real CLI verb or script**, not only a slash command. Agent slash commands are good UI; they should not be the only implementation of a workflow. (porting doc, "Replace Claude slash commands with durable commands")

Axis	Claude Code	Codex CLI	AIDA-style substrate	Default for a reversible setup
Skills / plugins	Large, fast-moving skill + plugin ecosystem; progressive disclosure	Codex skills (a `SKILL.md` directory) and plugins; a native `migrate-to-codex` skill imports Claude setup	Reusable workflow lives as docs + CLI verbs; only genuinely-reusable agent workflows become skills	Keep skills as **convenience over a CLI/MCP verb that already works**. Most workflow value should be in the verb, so it survives whichever agent (or no agent) runs it. (parity inventory, section 3)
MCP	First-class MCP client (`.mcp.json`)	First-class MCP across stdio and HTTP, OAuth/bearer; `codex mcp add`	MCP server is the substrate's typed cross-vendor surface (the token-efficient CLI is the primary agent surface; MCP costs ~2× for equal results — bench/agent-surface, SPIKE-73)	**MCP is the vendor-neutral layer.** This is the seam to standardize on regardless of agent. Register the same MCP server with each agent; the tool surface is identical. (porting doc + parity inventory both lean on this)

Axis	Claude Code	Codex CLI	AIDA-style substrate	Default for a reversible setup
Subagents	Subagents + a persistent named-team UX	Subagents (`.codex/agents/*.toml`, built-ins, `max_threads`/`max_depth`, Sub-agentStart/Stop hooks) as of 2026 — fan-out/delegation ports; the standing-team *UX* does not port one-to-one	Roles + leases coordinate work across sessions and vendors, a different (longer-lived) unit than within-session subagents	If you need within-session fan-out, both agents now do it. If you need cross-session, cross-vendor coordination of who-owns-what, that is the substrate's job, not a subagent's. (porting doc, "What changed recently")
Non-interactive automation	`claude -p` headless; hook-mediated `ask`/`defer` flows	`codex exec` (with `--json` event stream, `--output-schema`, explicit `--sandbox` + `--ask-for-approval`)	The orchestrator/drain that *invokes* the agent headlessly; the durable state the run reads and writes	For scripted runs, design an **explicit orchestrator checkpoint** for human approval — do not assume a stopped run leaves a resumable pending tool call. Externalize "needs a decision" as durable state. (porting doc, "Plan for non-interactive differences")

Axis	Claude Code	Codex CLI	AIDA-style substrate	Default for a reversible setup
Workflow state	Lives in Claude memory, hooks, and `.claude/` conventions unless deliberately externalized	Lives in `AGENTS.md`, Codex config, and `.codex/` conventions unless deliberately externalized	Git-canonical spec graph + queue + leases + briefs + punts + findings, readable by any MCP client	The portability question lives here. If load-bearing process state is trapped in one vendor's local conventions, switching agents becomes a workflow-portability problem, not a CLI swap. Push durable state into git/MCP/docs. (porting doc, "What AIDA Offers During This Migration")

Convenience vs load-bearing: the partition that actually matters

Before any migration or adoption decision, split every agent feature you rely on into two buckets. This partition does more work than the vendor comparison.

Load-bearing — it prevents unsafe work, enforces a lifecycle transition, gates a merge, or routes an approval. If this feature disappeared, a correctness guarantee would disappear. Examples from the source docs: a force-push guard, a commit-message-shape check, an advisor-role write block, an approval gate before a destructive action.

Convenience — shortcuts, status display, prompt sugar, UX. If it disappeared, you would be annoyed, not unsafe. Examples: a command-backed status footer, slash-command muscle memory, a skill that wraps a CLI verb you can already run by hand.

The rule that falls out of this partition:

> Retire convenience once an equivalent exists. Re-home load-bearing enforcement to the most reliable boundary available — sandbox/permission profiles,

git hooks, CI, a task runner, or an MCP server — *before* declaring any migration complete. Never re-emulate a load-bearing guarantee with prompt text.

This is the porting doc's central thesis ("Migrate workflows, not vibes"), and it holds whether or not you ever adopt a substrate. A team that does only this — and nothing else on this page — has already removed most of its agent lock-in.

Future-capability and lock-in trade-offs

Both Claude Code and Codex CLI ship quickly, and not in identical directions. That asymmetry is itself a decision input.

- **Betting exclusively on Claude Code** buys deeper Claude-native runtime features (the `defer`/resume primitive, the command-backed status line, the most mature skill ecosystem) at the cost of more vendor-specific coupling.
- **Betting exclusively on Codex CLI** buys a strong OpenAI-native execution environment (the strongest native sandbox of the two, clean non-interactive automation, `AGENTS.md` as a cross-vendor instruction surface) while giving up some mature Claude runtime affordances today.
- **A reversible bet** keeps project invariants *outside* either agent: scripts, MCP servers, CI, git hooks, orchestrators, and durable docs. You can then run either agent — or both, or a future third — against the same invariants, and a vendor switch costs you adapters, not correctness.

The reversible bet is the default recommendation of every source document, and it is available **without** adopting a substrate. `AGENTS.md` + CI + git hooks + careful sandbox/approval config + registered MCP servers is a complete, portable posture for many teams.

When to stay in Claude Code

- You actively depend on a Claude-only primitive: pending-tool-call `defer`/ resume, the command-backed status line, or deep Claude-native lifecycle automation, and rebuilding it elsewhere is not worth it yet.
- Your team's muscle memory and skill ecosystem investment is in Claude Code and there is no portability or sandbox pressure forcing a move.
- You are single-vendor by choice and have no plan to run a second agent against the same project.

In this case, the only homework is the reversibility test: make sure nothing *load-bearing* secretly depends on a Claude-only surface. If it does, re-home that one thing; otherwise stay put.

When to use (or move to) Codex CLI

- You want a stronger native OS sandbox over the whole process tree, with egress blocked by default — without building and maintaining your own outer wrapper.
- You want clean non-interactive automation with a structured event stream (`codex exec --json`) and an enforceable final-output schema.
- You are already in the OpenAI platform and value the native integration.
- You can accept today's gaps: no command-backed status line, no tool-call `defer`, no custom slash-command files. The source docs show each of these has a concrete workaround (footer fallback, park-and-resume, CLI verbs / skills).

The migration is real work but bounded, and a native `migrate-to-codex` skill bootstraps it. Treat that import as a starting point, not the finish line: the finish line is "the old workflow can be run, observed, interrupted, and recovered under the new agent's semantics."

When to adopt a vendor-neutral substrate (and when NOT to)

A substrate (AIDA or otherwise) is the right call only when a *specific* problem shows up that lighter tools do not solve:

- multiple agents or humans work in the same repo and step on each other;
- work loses context between sessions and across vendors;
- load-bearing process rules currently live trapped inside one agent's hooks, skills, or memory, and a vendor switch would lose them;
- reviews need to know which requirement a code change implements, and you want traceability from task to code to PR;
- you need an MCP-accessible coordination layer shared by Claude, Codex, and other tools;
- a future vendor switch is plausible and you want less agent lock-in.

If several of those are true, a substrate's durable, queryable, git-canonical state can be worth its overhead.

Honest "do NOT adopt a substrate when…" rows

These are the rows that make the matrix credible. For many readers one of these is the right answer, and the recommendation is to *not* take on the machinery.

- **One person, one repo, simple tracking.** A solo developer with simple issue tracking does not have the coordination problem a substrate solves. Use plain git plus a `TODO.md`, or first-feature scaffolding, and stop. The graph never gets big enough to pay for itself. (porting doc, "When not to add AIDA")
- **The agent was just an interactive coding assistant.** If you used Claude Code mostly for ad-hoc edits and there are no durable specs, lifecycle states, or cross-agent handoffs to preserve, there is nothing for a substrate to carry. Write a good `AGENTS.md`, move important command behavior into scripts, configure permissions, register any MCP servers, and verify with one real task. (porting doc, "The practical adoption path")

- **Single-vendor with no portability need.** A substrate's load-bearing differentiator is vendor-neutrality. If you are all-in on one agent and will never run a second vendor against the same specs, that differentiator does not pay — lean into the vendor's native coordination instead. (cross-vendor test)
- **Existing tools already carry the process clearly.** If GitHub Issues, Linear, Jira, or plain scripts already make the workflow legible enough, adding a second source of truth is cost, not clarity. The decision rule: if the problem is "the agent needs instructions," that is an `AGENTS.md` problem, not a substrate problem. (porting doc, "The decision rule is simple")
- **The goal is only "use a different agent for ad-hoc edits."** Switching agents is not a reason to adopt a substrate. Do the migration with `AGENTS.md`, scripts, sandbox config, and MCP registration; reach for a substrate only if a *separate* portability or coordination problem surfaces. (porting doc, "Do not add AIDA just because you are trying Codex")
- **You want zero discipline, not just zero infrastructure.** A substrate is low-infrastructure but not low-discipline — it asks for the habit of filing the spec, dropping the trace comment, naming the spec in the commit. If you will not keep even a hand-written `REQUIREMENTS.md`, the graph will not maintain itself. (`when-not-to-use-aida.md`, case 5)

A note on register: the substrate discussed here (AIDA) is itself a research probe and a work in progress — more mature in direction than in polish. That is exactly why the right default for most readers is the lighter path, and why a matrix that sends some readers *away* from it is the trustworthy one. If you do not have the portability or coordination problem, you do not need the substrate; say so to yourself before adopting it.

The default recommendation, in one rule

For a reversible migration, **keep your invariants outside vendor-specific agent runtimes.** Concretely:

1. Partition every agent feature into load-bearing vs convenience.
2. Re-home load-bearing enforcement to sandbox/permissions, CI, git hooks, task runners, or an MCP server.
3. Move reusable workflows into durable scripts or portable skills.
4. Move repo instructions into `AGENTS.md` (plus `CLAUDE.md` only as optional Claude-specific support).
5. Standardize coordination on MCP, the one layer both agents read identically.
6. Adopt a substrate **only** if a real coordination or portability problem remains after the steps above — and not before.

Do that, and the choice between Claude Code and Codex CLI shrinks from a high-stakes lock-in decision to a reversible tooling preference. Which is the point: the goal is not to pick the winning agent, it is to make the choice cheap to change.

See also

- `docs/agents/porting-claude-code-to-codex.md` — the full conceptual migration analysis this page distils, including the worked before/after examples.
- `docs/agents/claude-surfaces-codex-parity.md` — the surface-by-surface inventory with file/symbol references and per-axis coverage.
- `docs/plans/2026-06-25-codex-defer-resume-design.md` — the escalation/resume design behind the approval/defer row.
- `docs/plans/2026-06-25-codex-sandbox-posture-design.md` — the sandbox design behind the sandboxing row.
- when-not-to-use-aida.md — the broader honest scope limits for the substrate itself.
- README.md — the one-neighbor-at-a-time positioning index.